

Notes de cours

Introduction aux Langages et Automates (R4.12)

Christian Attiogbé

Nantes, Décembre 2022 (Version 0, en évolution, maj janvier 2025)



BUT2 - Informatique / IUT de Nantes

Table des matières

1	Introduction aux problèmes, langages formels et automates	5
2	Alphabet - Mot - Langage - Grammaire	9
2.1	Alphabet - Mot	9
2.2	Langages formels	11
2.3	Grammaires	13
2.4	Types et classification des grammaires : NOAM CHOMSKY . .	16
3	Langages rationnels/réguliers (Type 3) et Automates finis	19
3.1	Grammaires régulières et langages réguliers	19
3.2	Expressions régulières	20
3.3	Automates finis non-déterministes et déterministes	21
3.3.1	Automates finis déterministes	22
3.3.2	Les (automates) accepteurs	23
3.3.3	Automates finis non-déterministes	24
3.4	Théorème de KLEENE	26
3.5	Minimisation et autres opérations sur les automates	26
3.5.1	Minimisation : algorithme	26
3.5.2	Détermination AFND - AFD : algorithme	27
3.5.3	Produit d'automates : algorithme	27
3.5.4	Composition parallèle d'automates	28
3.6	Architecture d'un compilateur	29
3.6.1	Analyseur syntaxique	29
3.7	Ouvertures : Xtest	29
3.8	Machines à états de MEALY	30
3.9	Généralisation des machines à états	30
4	Langages non-contextuels (Type 2)	33
4.1	Langages non-contextuels - grammaires et automates à pile . .	33

Le cadrage du programme national (PN) du BUT Info.

Ressource R4.Real.12 : Automates et Langages

Compétence ciblée (C2)

- Proposer des applications informatiques optimisées en fonction de critères spécifiques : temps d'exécution, précision, consommation de ressources.

SAÉ au sein desquelles la ressource peut être mobilisée et combinée

- SAÉ 4.Real.01 - Développement d'une application complexe

Descriptif :

L'objectif de cette ressource est de voir les techniques liées aux automates utilisées en programmation.

Savoirs de référence étudiés :

- Langages rationnels et expressions régulières
- Automates finis déterministes
- Introduction aux grammaires
- Applications (par ex. : modélisation de systèmes simples, automates cellulaires...)

Prolongements suggérés :

- Automates finis non-déterministes

Apprentissages critiques ciblés :

- AC22.01 - Choisir des structures de données complexes adaptées au problème
- AC22.02 - Utiliser des techniques algorithmiques adaptées pour des problèmes complexes (par ex. recherche opérationnelle, méthodes arborescentes, optimisation globale, intelligence artificielle...)

Mots clés : Automates - Langages - Modélisation

Bibliographie Il existe de nombreuses ressources pédagogiques (livres, cours, sites web) sur la théorie des langages et des automates.

Nous conseillons pour cet enseignement les références suivantes :

- John E. Hopcroft, Rajeev Motwani et Jeffrey D. Ullman,
Introduction to Automata Theory, Languages, and Computation,
Addison Wesley, 2001, 2e éd., 521 p. (ISBN 978-0-201-44124-6 et
0201441241).
- Jacques Stern
Fondements mathématiques de l'informatique
Ediscience international, 2003 (Chapitre 1)
- Yliès Falcone, Jean-Claude Fernandez
*Automates à états finis et langages réguliers Rappels des notions essen-
tielles et plus de 170 exercices corrigés*
Dunod, 2020
- Compilateurs : Principes, techniques et outils
Alfred AHO, Ravi SETHI et Jeffrey ULLMAN.
Dunod, 2000. (vers anglaise *Compilers*, Addison Wesley, 1986)
- Alfred Aho, Monica Lam, Ravi Sethi et Jeffrey Ullman
*Compilateurs : principes, techniques et outils : Avec plus de 200 exer-
cices*
Pearson, 2007, 2e éd., 928 p. ISBN 978-2-7440-7037-2
- Peter Linz,
An Introduction to Formal Languages and Automata,
Jones & Bartlett Learning, 2001, 410 p, ISBN 978-0-7637-1422-2.

et des polys de cours

- Cours de Anca Muscholl, <https://www.labri.fr/perso/anca/Langages/cours/>, Université de Bordeaux
- Cours de Gérard Duchamp, Institut Galilée
[https://lipn.univ-paris13.fr/~duchamp/Enseignement/L3_Info/
Supports/Cours/THL_2.pdf](https://lipn.univ-paris13.fr/~duchamp/Enseignement/L3_Info/Supports/Cours/THL_2.pdf)
- Théorie des automates et langages formels
Michel Rigo
http://www.discmath.ulg.ac.be/cours/main_autom.pdf
- Notes de révision : Automates et langages
Benjamin MONMEGE et Sylvain SCHMITZ
http://www.lsv.fr/~schmitz/teach/2013_agreg/notes-r62.pdf
- Notes de cours, de Christine Solnon
<http://perso.citi-lab.fr/csolnon/langages.pdf>

— Cours de Sandrine Julia, Université de Nice
<http://deptinfo.unice.fr/~julia/AL>

Chapitre 1

Introduction aux problèmes, langages formels et automates

Au cœur des fondements de l'Informatique et de la programmation se trouve la manipulation des mots, des langages, des problèmes, des algorithmes.

Ces concepts sont étroitement liés entre eux et il convient de connaître ces liens/rerelations pour démêler plus facilement certains problèmes d'analyse, de raisonnement, de codage et de programmation.

Dans cet enseignement nous donnons une introduction qu'on pourra approfondir à l'aide de la littérature abondante (ouvrages, cours, logiciels, ...) qui sont accessibles sur le sujet.

La Théorie des Langages, les Automates et la Compilation sont parmi les domaines les plus rigoureusement étudiés et maîtrisés en Informatique. Ils ont de nombreuses applications et sont à la base de nombreux outils (compilateurs/interpréteurs, environnements de développement intégrés, débogueurs, etc).

Problème - mot - langage

Quelle est la relation entre les deux questions suivantes ?

1. Est-ce que 19 est paire ?
2. Est-ce que un entier n est paire ?

La question (1) est une instance de la (2)! qui a donc de nombreuses instances.

Du point de vue Informatique, on définit un **problème** comme la donnée d'**une entrée** appartenant à un ensemble, et d'**une propriété/question**.

Le **problème** consiste alors à **décider** si l'entrée vérifie la propriété ; ce qui revient à savoir s'il fait partie du sous-ensemble d'éléments ayant la propriété.

Si le problème est résolu par une machine (ordinateur), celle-ci va considérer, pour traiter le problème, une **représentation/encodage** de l'entrée (comme un **mot**).

Exemple

Problème

- $n=17$
- n est-il impaire ?

Décision

17 est encodé par 10001.

10001 fait partie de l'ensemble des entiers impairs ; leur représentation binaire se termine par 1 ; c'est-à dire, ils ont tous codés par **(0|1)*1**

Oui, 17 est impaire

En général, en Algorithmique/Programmation (Informatique), on répond plutôt à la question (2). On répond **oui** ou **non** ; donc on décide. La solution à un problème de décision, s'applique plus facilement aux instances du problème.

Ainsi, résoudre un problème consiste à reconnaître si un mot donnée (comme entrée du problème) est un élément de l'ensemble des mots satisfaisant la propriété/question ; un **ensemble de mots** constitue dans ce contexte un **langage formel** ; on dit aussi qu'un langage engendre un ensemble de mots.

Introduction : mot - langage - automate

Un automate est une machine (ou un algorithme, ou une procédure) qui permet de reconnaître les mots engendrés par un langage / décider si un mot appartient à un langage.

Exemple

Problème

- $Pgm = \text{product}(a,b) \{ \text{int } p = a; \text{ int } i = 0; \text{ while } (i \leq b) [\text{aaachoum !}] \{ i++; p = p + a \} \}$
- Pgm est-il syntaxiquement correct en Java ?

Décision

Le compilateur (= algorithme qui résout le problème) décide si Pgm ci-dessus fait partie d'**un des mots engendrés par le langage Java**.

Il y a plusieurs familles de problèmes :

- des problèmes de **calcul ou d'évaluation** ; ce sont ceux pour lesquels on effectue un calcul ou un traitement des données en entrée du problème ;
par exemples la factorielle de n , le maximum d'un ensemble de nombres, le tri d'un ensemble de données, ...
- des **problèmes de décision** ; ce sont ceux pour lesquels on peut répondre par oui ou par non ? (c'est vérifier si une donnée a une propriété)
Par exemple, un nombre est-t-il premier ? deux nombres sont-ils premiers entre eux ? une structure est-elle ordonnée ? ...
- des **problèmes d'énumération** ; par exemple, quels sont tous les sous-ensembles d'un ensemble d'éléments.
- les **problèmes d'optimisation** ;
- etc

Chapitre 2

Alphabet - Mot - Langage - Grammaire

Ce chapitre introduit les principaux concepts de la **théorie des langages** et des **automates**. Nous commencerons par le concept central de **mot** pour arriver à celui de automate.

2.1 Alphabet - Mot

Définition 1 *Alphabet.* Un alphabet (en théorie des langages) est un **ensemble fini** non vide de **symboles**.

Par convention, on note $\mathcal{A}$ l'alphabet ¹.

Terminologie Symbole = lettre, caractère, signe, ...

Exemples d'alphabet :

- $\mathcal{A} = \{0, 1\}$
- $\mathcal{A} = \{a, z, i\}$
- $\mathcal{A} = \{-, ;, :, =,], [,), (, \}$
- $\mathcal{A} = \{co, ri, ca, ci\}$
- $\mathcal{A} = \{end, begin, while, done, break, wait\}$

Définition 2 *Mot.* Un mot w est une suite finie d'éléments (ou symboles) définis sur un alphabet $\mathcal{A}$ donné. Un mot est donc défini sur $\mathcal{A} \times \mathcal{A} \times \mathcal{A} \times \dots$ (on verra la notation appropriée $\mathcal{A}^* = \mathcal{A} \times \mathcal{A} \times \mathcal{A} \times \dots$ dans la suite).

1. On trouve aussi dans la littérature, la notation Σ pour l'alphabet

Le **mot vide** est autorisé et est noté ϵ

Exemples de mots :

— 000001 011011001 1010101010 111 10001
 — iza azi aziza izazi iizzzaaaaaa zazizaza
 — ;-) :=) :- ([: :(; :-(:-))))) [-] ;-]
 — cocorico cacorici caricoco cacorici
 — begin end end end begin
 while begin end wait while begin
 done break while end while while end end

Longueur d'un mot : la longueur d'un mot w est le nombre de symboles qui composent le mot ; c'est un entier nul ou positif. On note $|w|$ la longueur de w .

$\mathcal{A}^*$ représente les mots de **longueur quelconque y compris le mot vide** (ens. des mots de longueur ≥ 0)

$\mathcal{A}^+$ représente les mots constitués d'**au moins un symbole** ; ce sont les mots de longueur supérieure ou égale à 1.

$$\mathcal{A}^+ = \{w \mid w \in \mathcal{A}^* \text{ et } |w| \geq 1\}$$

$$\mathcal{A}^* = \mathcal{A}^+ \cup \{\epsilon\}$$

Opérations sur les mots : concaténation, préfixe, suffixe, facteur

Etant donné deux mots, w_1 et w_2 , on peut concaténer w_1 et w_2 pour former un nouveau mot w , constitué de w_1 **suivi de** w_2 , et de longueur $|w| = |w_1| + |w_2|$, avec $|\epsilon| = 0$.

L'opération de **concaténation** est souvent notée $\cdot$.

Mais on peut ne pas le faire apparaître quand il n'y a pas d'ambiguïté. Notons que la longueur d'un mot ne tient pas compte de l'opérateur $(\cdot)$ de concaténation.

Exemples

La concaténation :

jour.bon donne le mot jourbon
 bon.jour donne le mot bonjour
 bonjour. ϵ donne le mot bonjour

La concaténation peut se faire à droite ou à gauche d'un mot ; mais la concaténation **n'est pas commutative** ; elle est **associative** ; le mot vide ϵ est un élément neutre pour la concaténation.

On dit que $(\mathcal{A}, .)$ forme un *monoïde*.

On peut concaténer un même mot w autant de fois (en le précisant) qu'on veut ; par exemple, si $w = \text{til}$, alors $w^3 = (w)^3 = \text{tiltiltil}$.

Exemples

$\text{jour.bon} \neq \text{bon.jour}$

$(\text{bon.jour}).\text{bahia} = \text{bon.}(\text{jour.bahia})$

Il est possible que les symboles de l'alphabet ne soit plus distinguable une fois des mots concaténés.

De la même façon on peut extraire les préfixe et suffixe d'un mot, en précisant leurs longueurs. On peut décider si un mot est préfixe ou un suffixe d'un autre.

Un mot p est **préfixe** d'un mot w sur un alphabet $\mathcal{A}$ s'il existe un mot r tel que $p.r = w$

On définit le **suffixe** de façon similaire.

Les chaînes de caractères ou *string* en programmation sont un exemple trivial des mots.

2.2 Langages formels

Un langage permet d'utiliser des mots pour exprimer des idées, des intentions, des concepts, des réalités de presque tout.

Définition 3 *Langage.* Un langage $\mathcal{L}_A$ défini sur un alphabet $\mathcal{A}$ est un sous-ensemble de mots de $\mathcal{A}^*$:

$$\mathcal{L}_A \subseteq \mathcal{A}^*$$

Opérations sur les langages. Les langages étant des ensembles de mots, on définit simplement plusieurs opérations ensemblistes sur les langages.

Union de langages L'union de deux langages, $\mathcal{L}_1$ défini sur un alphabet $\mathcal{A}_1$ et $\mathcal{L}_2$ défini sur $\mathcal{A}_2$ est le langage défini par l'ensemble des mots appartenant à $\mathcal{L}_1$ ou à $\mathcal{L}_2$:

$$\mathcal{L}_1 \cup \mathcal{L}_2 = \{w \mid w \in \mathcal{L}_1 \vee w \in \mathcal{L}_2\}$$

Intersection de langages L'intersection de deux langages $\mathcal{L}_1$ défini sur un alphabet $\mathcal{A}_1$ et $\mathcal{L}_2$ défini sur $\mathcal{A}_2$ est le langage défini par l'ensemble des mots appartenant à la fois à $\mathcal{L}_1$ et à $\mathcal{L}_2$:

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \{w \mid w \in \mathcal{L}_1 \wedge w \in \mathcal{L}_2\}$$

Différence de langages ...

$$\mathcal{L}_1 / \mathcal{L}_2 = \{w \mid w \in \mathcal{L}_1 \wedge w \notin \mathcal{L}_2\}$$

Produit de langages

$$\mathcal{L}_1 . \mathcal{L}_2 = \{w \mid \exists m \in \mathcal{L}_1 \wedge \exists v \in \mathcal{L}_2 \wedge w = m.v\}$$

Exemple :

$$\{a, b\} . \{0, 1\} = \{a0, a1, b0, b1\}$$

Quotient de langages $\mathcal{L}_2$ par $\mathcal{L}_1$

$$\mathcal{L}_1^{-1} . \mathcal{L}_2 = \{v \mid \exists m \in \mathcal{L}_1 \wedge m.v \in \mathcal{L}_2\}$$

Exemple :

$$\{a, b\}^{-1} . \{a0, a1, b0, b1\} = \{0, 1\}$$

Puissance de langages Puissance n du langage $\mathcal{L}$. Elle est définie inductivement :

$$\begin{aligned} \mathcal{L}^0 &= \{\epsilon\} \\ \mathcal{L}^{n>0} &= \mathcal{L} . \mathcal{L}^{n-1} \end{aligned}$$

Exemple :

$$\mathcal{L} = \{0, 1\}, \quad \mathcal{L}^2 = (\{0, 1\})^2 = \{00, 01, 10, 11\}$$

Clôture/fermeture (de Kleene) de langage

$$\mathcal{L}^* = \bigcup_{i \geq 0} (\mathcal{L}^i)$$

$$\mathcal{L}^+ = \mathcal{L}^n \setminus \{\epsilon\}$$

Les langages formels sont de divers types. Cela dépend de leurs caractéristiques, propriétés, construction. Par exemple :

Langages rationnels/réguliers (*regular languages*). Un langage est rationnel s'il et seulement si il est dénoté par une *expression régulière*.

On y revient dans la section 2.3.

Constats. On voudrait bien former des mots spécifiques, et faire des suites de mots spécifiques (donc faire des phrases = une structure particulière). Cela revient à se donner des **règles de formation des mots** et des langages.

2.3 Grammaires

Un langage peut être contraint par des règles particulières pour structurer la suite de mots de l'alphabet considéré.

Une grammaire est un ensemble de règles qui engendre un langage. Il est formellement défini comme suit.

Cependant, toutes les phrases construites en respectant les règles de grammaire d'un langage n'ont pas forcément de sens. On peut exprimer des phrases sans tête ni queue ; par exemple :

le berger gerbé a mergé en émergeant !

genre tête carrée.

On peut aussi exprimer une même idée de différentes façons dans un même langage.

Contrairement au langage naturel, il n'y a à priori pas d'ambiguïté dans le langage formel.

Définition 4 *Grammaire.* Une grammaire (notée G) est un quadruplet $\langle T, N, S, R \rangle$ avec

- T est un alphabet, appelé le **vocabulaire terminal** ; ce sont les éléments qui vont apparaître dans le langage engendré.

- N est un ensemble de symboles, appelé le **vocabulaire non terminal**, qui n'apparaîtront pas dans les mots générés, car ils seront remplacés par des terminaux; ils sont utilisés pendant la génération.
- S est le symbole de départ ou **axiome** de la génération du langage. S est un élément de N . La génération des mots commencera par S en utilisant des règles pour produire des transformations de S et d'autres éléments de N qui vont être introduits.
- R est un ensemble de règles dites **règles de production**. Les règles de production ont la forme : $L \rightarrow R$ où

$$L \in (N \cup T)^+ \quad \text{et} \quad R \in (N \cup T)^*$$

Les règles permettent de produire progressivement la structure ou la composition des mots du langage voulu.

On appelle aussi un symbole **non terminal**, une “catégorie syntaxique”. Les éléments de $R \in (N \cup T)^*$ sont appelés des **formes**. Une forme peut être transformée en une autre en appliquant une règle de production.

Une règle de la forme $L \rightarrow T^*$ est appelée **une règle terminale** (on ne pourra plus transformer la partie droite).

Exemple de grammaire

$$\begin{aligned}
 G_1 = \langle & \\
 & T = \{x, ;, -, +, *, 2, 7\}, \\
 & N = \{V, E, P, I, OP, SI\}, \\
 & S = P, \\
 & R = \{P \rightarrow I SI \\
 & \quad SI \rightarrow ; I SI \\
 & \quad SI \rightarrow \epsilon \\
 & \quad I \rightarrow V = V OP E \\
 & \quad V \rightarrow x \\
 & \quad OP \rightarrow + \mid - \mid * \\
 & \quad E \rightarrow 2 \mid 7 \mid V OP E \\
 & \quad \} \\
 & \rangle
 \end{aligned}$$

$$\begin{array}{l}
G_2 = \langle \\
\quad T = \{a, b, z, f, v, i, o, n, t\}, \\
\quad N = \{P, M, I\}, \\
\quad S = P, \\
\quad R = \{P \rightarrow aMa \\
\qquad\qquad M \rightarrow bIb \mid zIz \mid fIf \mid nIn \mid tIt \mid vIv \\
\qquad\qquad I \rightarrow i \mid o \\
\qquad\qquad \} \\
\quad \rangle
\end{array}$$

Dérivation Une forme non vide g peut se transformer par une grammaire en une forme éventuellement vide d . On dit qu'il y a **dérivation de g en d en une étape**, et on note cela $g \Rightarrow d$, lorsqu'il existe une règle de la grammaire G qui permet de transformer g en d (en tenant compte des préfixes ou suffixes éventuels de g et d).

Par généralisation, on effectue **des dérivations en une ou plusieurs étapes** (éventuellement vides) : $g \Rightarrow^+ d$ ou $g \Rightarrow^* d$.

Une grammaire engendre un langage : l'ensemble des mots obtenus en partant de l'axiome de la grammaire et en appliquant les règles de production, jusqu'à ne plus avoir de non terminaux dans les mots.

Langage engendré par une grammaire. Le langage engendré par une grammaire $G = \langle T, N, S, R \rangle$ est l'ensemble des mots (terminaux = dans T) obtenus par dérivations successives à partir de l'axiome de G . On note ce langage $\mathcal{L}(G)$

$$\mathcal{L}(G) = \{w \mid w \in T^* \wedge S \Rightarrow^+ w\}$$

Exemples les **trois (3) mots** suivants sont engendrés par la grammaire G_1 donnée en exemple dans la section 2.3, page 13 :

$$\begin{array}{l}
x=x-7; x=x+x+2; x=x+x-x-x-7 \\
x=x-2; x=x-x-2; x=x+x-x-x^*7 \\
x=x^*7; x=x^*x-2; x=x^*x^*x^*x^*7; x=x-2
\end{array}$$

Les huit (8) mots suivant font partie des mots engendrés par la grammaire G_2 donnée en exemple dans la section 2.3, page 13 :

aziza, abiba
 aziza, anona, afifa
 azoza, afofa, aviva

Remarque : en fonction de la structuration des règles de production d'une grammaire, on a plus ou moins de restrictions pour la construction des mots, et ainsi différents types de grammaires.

2.4 Types et classification des grammaires : Noam Chomsky

Les restrictions imposées (par les concepteurs) sur la forme des règles de production des grammaires aboutissent à une classification et une hiérarchisation des grammaires. Ainsi quatre (4) types de grammaire ont été définis en 1957 par N. CHOMSKY.

Vous aurez remarqué l'importance et l'usage intensif des langages réguliers et des langages hors-contexte pour les langages de programmation habituels.

Hiérarchisation. Les langages engendrés par ces types de grammaire sont ordonnés par une inclusion stricte. Notons $\mathcal{L}_{G_{type_i}}$ les langages engendrés par une grammaire de type i . On a alors :

$$\mathcal{L}_{G_{type3}} \subset \mathcal{L}_{G_{type2}} \subset \mathcal{L}_{G_{type1}} \subset \mathcal{L}_{G_{type0}}$$

Type 0	Pour les grammaires de ce type, il n'y a aucune restriction sur la structure des règles. Les grammaires de ce type permettent d'engendrer tous les langages qui peuvent être reconnus en un temps fini, par une machine ; ils sont dit langages décidables.
Type 1	<i>Context-dependent grammar</i> ou grammaire dépendant du contexte ; ces grammaires sont dites sensibles au contexte ou contextuelles. Les règles de production (dans R) de ces grammaires sont de la forme : $cXc' \rightarrow cX'c'$. La partie gauche d'une règle spécifie un contexte de dérivation pour X, avec $c \in (N \cup T)^+$ et $c' \in (N \cup T)^+$. $X \in N$ et $X' \in (N \cup T)^*$ Ces grammaires engendrent les langages contextuels, dépendant du contexte.
Type 2	<i>Context-free grammar</i> ou grammaire hors-contexte ou grammaire non-contextuelle. Les règles dans R sont de la forme : $X \rightarrow X'$ avec $X' \in (N \cup T)^*$. Elles ne spécifient pas de contexte, le membre gauche est constitué d'un seul non terminal : $X \in N$. Ces grammaires engendrent les langages non contextuels (indépendants du contexte, hors-contexte).
Type 3	<i>Regular grammars</i> Grammaires régulières. Lorsque les règles de G sont de la forme $X \rightarrow \alpha.X'$ où $\alpha \in T$ et $X, X' \in N$ la grammaire est dite régulière à gauche. $X' \rightarrow \alpha \mid \epsilon$ autorisée. Lorsque les règles de G sont de la forme $X \rightarrow X'.\alpha$ où $\alpha \in T$ la grammaire est dite régulière à droite. Ces grammaires engendrent les langages réguliers (à gauche ou à droite). Les langages réguliers sont aussi appelés langages rationnels ou reconnaissables.

TABLE 2.1 – Synthèse types de grammaire (CHOMSKY)

Chapitre 3

Langages rationnels/réguliers (Type 3) et Automates finis

Nous avons vu dans le chapitre précédent qu'un langage régulier est un langage engendré par une grammaire régulière G (type 3).

Rappels : L est un **langage régulier** si et seulement si, il existe une grammaire régulière G qui engendre L on note $L = \mathcal{L}(G)$. Par abus, on utilisera $\mathcal{L}$ pour un langage.

3.1 Grammaires régulières et langages réguliers

Rappelons que lorsque les règles de production d'une grammaire G **sont de la forme** $X \rightarrow \alpha.X'$ où $\alpha \in T$ et $X, X' \in N$ alors la grammaire G est dite **régulière à gauche** (notons que $X \rightarrow \alpha|\epsilon$ est autorisé).

Lorsque les règles de G sont de la forme $X \rightarrow X'.\alpha$ où $\alpha \in T$ la grammaire est dite **régulière à droite**.

Cette distinction est importante lorsqu'on analyse une expression ou un mot (en le lisant de gauche vers la droite / de droite vers la gauche) ; on peut effectuer l'analyse en partant de l'axiome de la grammaire vers le mot à reconnaître, ou du mot à reconnaître vers l'axiome. C'est un exemple de procédure implantée dans la phase d'analyse syntaxique dans les compilateurs.

Propriétés des langages réguliers. L'union ($\cup$), l'intersection ($\cap$), le produit ($\times$), la fermeture ($*$) de langages réguliers sont aussi des langages

réguliers.

3.2 Expressions régulières

Définition 5 *Expression régulière* Une expression régulière E , sur un alphabet $\mathcal{A}$ est définie inductivement comme suit (avec les opérateurs $.$ $+$ $*$:

- $E = \epsilon$ (ou $E = \{\}$) ou
- $E = \alpha$ si $\alpha \in \mathcal{A}$ ou
- $E = E_1 + E_2$ avec E_1 et E_2 deux expressions régulières sur $\mathcal{A}$
- $E = E_1.E_2$ avec E_1 et E_2 deux expressions régulières sur $\mathcal{A}$ ou
- $E = E_k^*$ avec E_k une expression régulière sur $\mathcal{A}$
- $E = (E_k)$ avec E_k une expression régulière sur $\mathcal{A}$

On note les trois opérations régulières : concaténation, choix/union finie ($+$ ou $—$), itération.

Exemple 1 : Soit l'alphabet $\mathcal{A}_1 = \{a\}$. Voici des exemples d'expressions régulières suivantes sur $\mathcal{A}$:

$a.a$ $a+a$ $a.a.a.a.a$ $(a+b).aaaa$

Exemple 2 : Soit l'alphabet $\mathcal{A}_2 = \{li, lo, ok, ko, c1, c2, c4, pa\}$. Voici des exemples d'expressions régulières suivantes sur $\mathcal{A}$:

$lo.li.ok.pa.ko.c1$
 $li.pa.ko.pa.ok.c1.lo$

Remarque : Dans les expressions régulières, le $.$ est facultatif s'il n'y a pas d'ambiguïté.

L'opérateur $+$ se note aussi $|$ pour éviter l'ambiguïté avec avec le $+$ de la répétition.

On peut écrire une **infinité de mots** !

L'ensemble des langages réguliers/rationnels sur un alphabet $\mathcal{A}$ est infini dénombrable. L'ensemble des langages sur un alphabet $\mathcal{A}$ est infini.

L'équivalence entre expressions régulières et langages réguliers est établie par les deux théorèmes suivants.

Théorème

Toute expression régulière E_r décrit un langage régulier L_r :
 $(E_r \implies L_r)$

Théorème

Tout langage régulier L_r peut être décrit par une expression régulière
 $E_r : (L_r \implies E_r)$.

3.3 Automates finis non-déterministes et déterministes

Un automate est un modèle fondamental; il permet de modéliser, de raisonner, d'analyser, de simuler.

Notions de bases : état, transition, comportement

Etat initial (E_i), état final (E_f), transition (étiquetée avec **alpha**) entre états E_2 et E_3



Automates : définition

Automates à états

Un automate à états est défini formellement par un n-uplet :
 $\langle S, A, \delta, S_0, S_f \rangle$

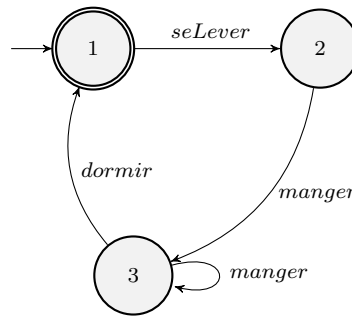
- Ensemble d'**états** fini : $S = \{S_0, \dots, \dots\}$
- Un alphabet d'**actions** (ou **étiquettes** ou **symboles**) : A
- Une relation de **transition** δ définie sur $(S \times A)$ et S comme suit : $\delta : (S \times A) \leftrightarrow S$
- Un **état initial** S_0 (c'est à dire $S \in S_0$)
- Un ou des **états finaux** S_f (c'est à dire $S_f \subseteq S$)

3.3.1 Automates finis déterministes

Automate fini déterministe

L'automate est **déterministe** lorsque il y a **un seul état initial** et la relation de transition δ est une **fonction** (et non une relation).

Exemple d'automate déterministe



Cet automate est décrit formellement par :

$$\begin{aligned}
 S &= \{1, 2, 3\} \\
 A &= \{seLever, dormir, manger\} \\
 \delta : (S \times A) &\leftrightarrow S \text{ et} \\
 \delta &= \{((1, seLever), 2), ((2, manger), 3), ((3, dormir), 1), \\
 &((3, manger), 3)\} \\
 S_0 &= 1 \\
 S_f &= \{1\}
 \end{aligned}$$

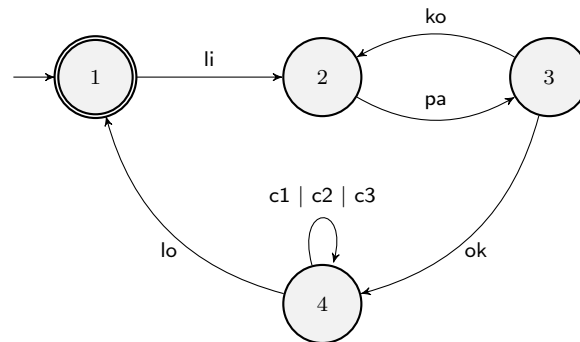
On peut représenter la relation δ par une matrice :

S \ A	seLever	manger	dormir
1	2		
2		3	
3		3	1

Exercice Représenter la relation par une matrice $(S \times S) \leftrightarrow A$

Exemple d'automate déterministe

3.3. AUTOMATES FINIS NON-DÉTERMINISTES ET DÉTERMINISTES 23



3.3.2 Les (automates) accepteurs

Algorithme d'un accepteur

```

entrée : un automate et une suite/chaîne de caractères :
sortie : Oui | Non
Début
   $Etatc \leftarrow S_0$  {partons de l'état initial}
  {On lit une suite/chaîne de caractères en entrée}
  {On prendra carCourant comme le caractère suivant
  dans la chaîne}
  while  $Etatc \neq S_f \wedge Etatc \neq EtatNonDef$  do
    lire carCourant {on parcourt la chaîne de car-
    ractères en entrée}
    afficher ( $Etatc$ , carCourant)
     $Etatc \leftarrow \delta(Etatc, carCourant)$ 
  end while
  if  $Etatc = S_f$  then
    Bien terminé : la suite de caractères en entrée est
    reconnue
  else {On s'est arrêté sur un état non défini}
    Echec : la suite de caractères n'est pas reconnue
  end if
  {Attention : algo à parfaire}
fin
  
```

On expérimentera cet algorithme en TD/TP.

Vous pouvez aussi vous amuser avec mes nombreux outils existants ; par exemple Aude (à <https://aude.imag.fr/>) ; FSM designer <https://madebyevan.com/fsm/>.

3.3.3 Automates finis non-déterministes

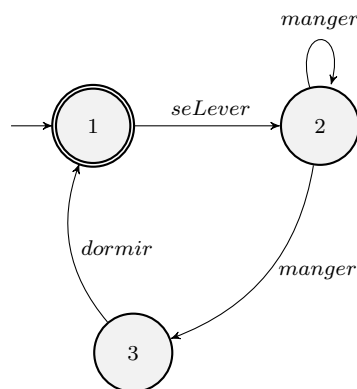
Automate fini non déterministe

L'automate est **non-déterministe** lorsque : il a plusieurs **états initiaux** ou la relation de transition δ est bien une **relation** (et non une fonction) - c'est à dire, partant d'un état et d'un symbole, il y a plusieurs états suivants.

Une transition sans étiquette de l'alphabet est implicitement étiquetée par le mo vide ϵ : il s'agit d'une ϵ -transition.

Un automate avec une ou des ϵ -transitions est un automate non déterministe.

Exemple d'automate non-déterministe



Ouvertures L'algorithme de THOMSON permet de construire un automate non-déterministe (avec des ϵ -transition) à partir d'une expression régulière.

Terminologie

- **Espace d'états** : ensemble des états (possibles) d'un système
- **Relation de transition, Fonction de transition**
- Automate (*Automaton, Automata*)
- Machine à états finis *Finite State Machine*)
- **Graphe d'états**
- Diagramme Etat/Transition (*State/Transition Diagram*)
- Système de transitions (étiquetées) *Labelled Transition System*
- Espace d'états (*state space*)

3.3. AUTOMATES FINIS NON-DÉTERMINISTES ET DÉTERMINISTES 25

Modélisation En modélisation et en construction de logiciels en particulier, on **prévoit le comportement des logiciels**, à l'aide de **modèles**. Les automates (avec les graphes) sont largement utilisés pour cela.

Le **comportement d'un système** (quel qu'il soit), peut être vu comme une suite des différents états du système.

Les changements d'états ou **transitions entre états** peuvent être internes ou dus à des événements externes au système.

Une **suite d'états décrit un comportement**, donc des transitions entre des états.

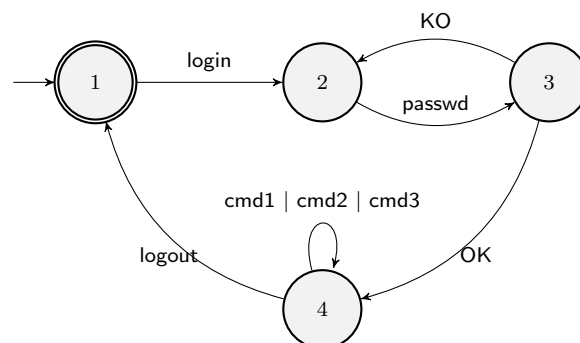
On **modélise ainsi ce qu'on va construire** (aussi bien les données que les traitements).

On peut ainsi **prédire les comportements corrects et incorrects** (*bugs*) ; il faut pour cela que les modèles soient (mathématiquement) les plus précis possibles.

Modélisation de système

Un modèle simule mathématiquement le (comportement d'un) système. En ce sens un automate à états sert de modèle d'un système ; il permet d'en simuler le comportement.

Exemple Modèle à états d'un processus client qui se connecte à un processus serveur (pour une interaction/coopération à la réalisation d'une tâche).



3.4 Théorème de Kleene

L'équivalence entre automates finis et langages réguliers est établi par deux (2) théorèmes.

Théorème 1 : Tout langage accepté par un automate fini est régulier.

Théorème 2 : Tout langage régulier est accepté par un automate fini.

Le théorème s'énonce comme suit :

Théorème de Kleene

Soit $\mathcal{A}$ un alphabet, pour tout langage L sur $\mathcal{A}^*$, (ou encore pour $L \subseteq \mathcal{A}^*$), les **deux assertions suivantes sont équivalentes** :

- L est un langage régulier,
- il existe un automate fini déterministe A qui reconnaît L , c'est à dire $L = \mathcal{L}(A)$

3.5 Minimisation et autres opérations sur les automates

3.5.1 Minimisation : algorithme

L'algorithme de minimisation prend en entrée un automate fini déterministe (complet = un nombre fini d'états) dont chaque état est accessible à partir de l'état initial, et donne en sortie un automate fini minimal (= très peu d'états) qui lui est équivalent.

Un langage rationnel est reconnu par un unique automate fini déterministe minimal.

Le principe de base de l'algorithme de minimisation est de

- fusionner (ou confondre) les **états équivalents**.
- séparer les états (partitionner) en distinguant les finaux ou les non finaux. Dans chacune des partitions, on sépare les états non équivalents.
- continuer ce processus jusqu'à ce qu'il se stabilise.

3.5. MINIMISATION ET AUTRES OPÉRATIONS SUR LES AUTOMATES 27

Etats équivalents Soit $A_1 = \langle S, A, \delta, S_0, S_f \rangle$ un automate déterministe. Deux états p et q de A_1 sont dits équivalents si les langages reconnus à partir de chacun des états sont les mêmes.

$$p \approx q \iff \mathcal{L}_p(A) = \mathcal{L}_q(A)$$

intuitivement, on fait les mêmes actions à partir de p ou de q .

Remarques. La relation $\approx$ est une relation d'équivalence. On note $[p]$ les états équivalents à p .

Soit $A_1 = \langle S, A, \delta, S_0, S_f \rangle$, l'automate minimal résultant de la minimisation de A_1 est $A_{1min} = \langle S', A', \delta', [S_0], S'_f \rangle$ tel que :

$$S' = \{[s] \mid s \in S\}$$

$$A' = A$$

$$\delta' = \{([s], \alpha, [t]) \mid \exists s' \in [s] \wedge \exists t' \in [t] \wedge (s', \alpha, t') \in \delta\}$$

$$S'_f = \{[s_f] \mid s_f \in S_f\}$$

Pratiquement, on construit progressivement/inductivement les classes d'équivalence sur S c'est à dire, on construit $\approx_0$ puis $\approx_1, \dots, \approx_i$, avec :

$$s \approx_0 t \text{ \{ première partition en finaux ou non finaux \}}$$

$$s \approx_0 t \iff ((s \in S_f \wedge t \in S_f) \vee (s \notin S_f \wedge t \notin S_f))$$

pour les étapes suivantes, on affine chaque partition :

$$s \approx_i t \iff \begin{cases} s \approx_{i-1} t \\ \wedge \\ \forall \alpha \in A. \delta(s, \alpha) \approx_{i-1} \delta(t, \alpha) \end{cases}$$

Exemples. Smileys (déjà vu), etc

Références Algo de Moore, Algo de Hopcroft, etc.

Ouvrage de Hopcroft, Motwani, Ullman; Ouvrage de Séébold; Notes de cours listés au début du cours. Et aussi comme point d'entrée https://fr.wikipedia.org/wiki/Minimisation_d%27un_automate_fini_d%C3%A9terministe

3.5.2 Déterminisation AFND - AFD : algorithme

3.5.3 Produit d'automates : algorithme

Soient deux automates $M_1 = \langle S_1, A, \delta, S_{01}, S_{f1} \rangle$ et $M_2 = \langle S_2, A, \delta, S_{02}, S_{f2} \rangle$;

Ici M_1 et M_2 ont le même alphabet. Autrement, on effectue l'union des alphabets ($A = A_1 \cup A_2$).

Le **produit synchrone** de M_1 et M_2 est l'automate $M = \langle \dots \rangle$ tel que :

- $S_1 \times S_2$, le produit cartésien des ensembles d'états,
- A , l'alphabet commun,
- δ_{12} , la relation de transition (les transitions communes à M_1 et M_2),
- (S_{01}, S_{02}) , le couple des états initiaux,
- $(S_{f1} \times S_{f2})$, le produit des états finaux.

La relation de transition δ_{12} est construite telle que :

$$((S_{i1}, S_{i2}), \alpha, (S_{j1}, S_{j2})) \in \delta_{12} \iff (S_{i1}, \alpha, S_{j1}) \in \delta_1 \wedge (S_{i2}, \alpha, S_{j2}) \in \delta_2$$

Remarque. Le langage engendré par le produit de deux automates est l'intersection des langages engendrés par chacun d'eux.

$$\mathcal{L}(M_1 \times M_2) = \mathcal{L}(M_1) \cap \mathcal{L}(M_2)$$

En effet en pratique, le produit d'automates est aussi une solution à la recherche de mots reconnus **à la fois** par les deux automates.

Exemple : écrire un automate reconnaissant sur l'alphabet $\{A, b\}$, les mots contenant une suite de A, et ne contenant pas deux b successifs.

3.5.4 Composition parallèle d'automates

Pour définir par exemple la sémantique de la concurrence entre des processus. Par exemple, l'interaction entre un processus client et processus serveur de ressources. Ces processus vont se synchroniser sur des actions communes (de leur alphabet), mais chacun d'eux fera indépendamment de l'autre les actions non communes.

Soient deux automates $M_1 = \langle S_1, A, \delta, S_{01}, S_{f1} \rangle$ et $M_2 = \langle S_2, A, \delta, S_{02}, S_{f2} \rangle$;
On effectue l'union des alphabets ($A = A_1 \cup A_2$).

Le **produit parallèle** de M_1 et M_2 est l'automate $M = \langle \dots \rangle$ tel que :

- $S_1 \times S_2$, le produit cartésien des ensembles d'états
- A , l'alphabet commun
- δ_{12} , la relation de transition de l'interaction entre M_1 et M_2
- (S_{01}, S_{02}) , le couple des états initiaux
- $(S_{f1} \times S_{f2})$, le produit des états finaux.

La relation de transition δ_{12} est construite telle que :
 $\forall \alpha \in A,$

$$\delta_{12}((S_{i1}, S_{i2}), \alpha) = \begin{cases} (S_{j1}, S_{j2}) & \text{if } \delta_1(S_{i1}, \alpha) = S_{j1} \wedge \delta_2(S_{i2}, \alpha) = S_{j2} \\ (S_{j1}, S_{i2}) & \text{if } \delta_1(S_{i1}, \alpha) = S_{j1} \wedge \delta_2(S_{i2}, \alpha) = \text{undef} \\ (S_{i1}, S_{j2}) & \text{if } \delta_1(S_{i1}, \alpha) = \text{undef} \wedge \delta_2(S_{i1}, \alpha) = S_{j2} \\ \text{undef} & \text{if } \delta_1(S_{i1}, \alpha) = \text{undef} \wedge \delta_2(S_{i2}, \alpha) = \text{undef} \end{cases}$$

3.6 Architecture d'un compilateur

Les compilateurs de langages de programmation (classiques) ont une architecture précise comme celle indiquée dans la figure 3.6.

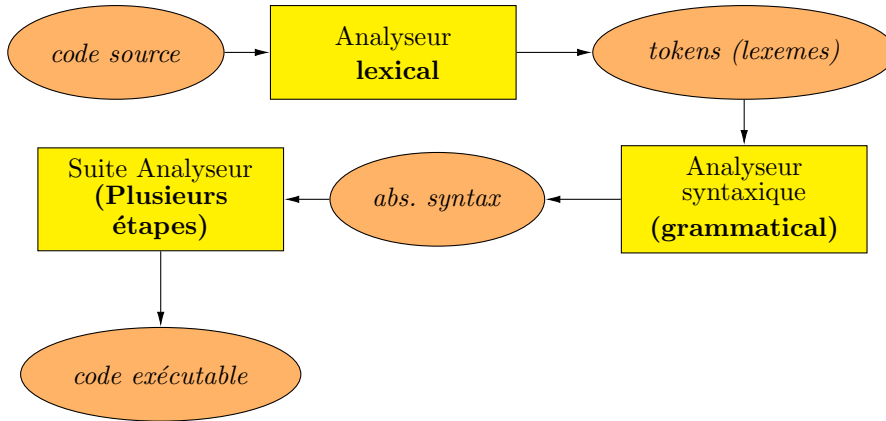


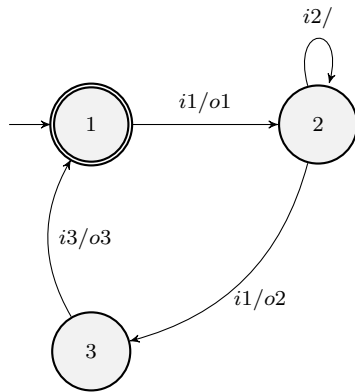
FIGURE 3.1 – Schéma du processus de compilation

3.6.1 Analyseur syntaxique

3.7 Ouvertures : Xtest

3.8 Machines à états de Mealy

Machine à états de Mealy Un automate de Mealy est défini par un n-uplet $\langle S, A = In \cup Out, \delta, \delta_o, S_0, S_f \rangle$



- Ensemble d'états : S
- Ensemble d'entrées : In
(événements d'entrées, conditions)
- Ensemble de sorties : Out
(actions de sortie, traitements)
- Un état initial : S_0
- **Deux relations(fonctions) :**
 - transition $\delta : S \times In \rightarrow S$
 - **sortie** $\delta_o : S \times In \rightarrow Out$
- Etat final S_f

Machines de Mealy : notation

- Notation des transitions : $S_s \xrightarrow{i/o} S_t$
- si l' **entrée i est reçue** alors que le système est dans l'état S_s , la **sortie o est produite** et le nouvel état du système est S_t
- i est aussi appelé le déclencheur (*trigger*).

La notation des transitions est étendue de la façon suivante : les transitions entre états ont la forme **evt [garde] / actions***.

evt est l'entrée reçue/lue.

[garde] : garde est une condition sur l'état du système après l'événement evt.

action* est une suite de 0 ou plusieurs actions.

Exemple d'automate de Mealy

3.9 Généralisation des machines à états

Il y a plusieurs modèles et familles de modèles à états. Plusieurs langages ou formalismes de modélisation utilisent ces modèles à états.

- **System Design Language (SDL)** (normalisé par la *International Telecommunication Union* (ITU), 1988, très utilisé en Télécoms et ailleurs)

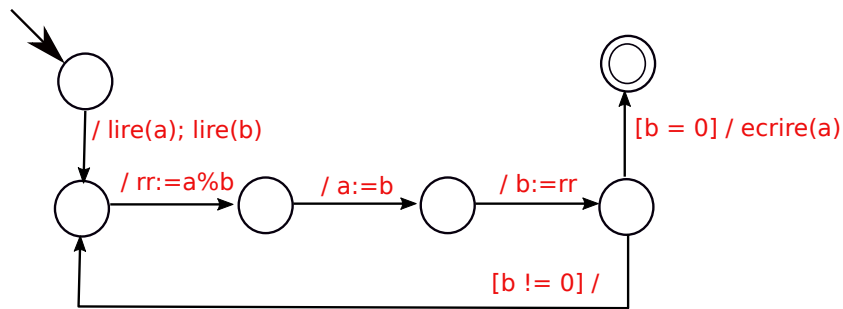


FIGURE 3.2 – Exemple de modèle

- **Statecharts** de David Harel (hiérarchiques) 1987, utilisé par exemple dans papyrus, ...
- **Réseaux de Petri** de Carl Adam Petri (1962, 1969)
- **Algèbres de processus**, telles que *Calculus of Communicating Systems* (CCS, 1980) de R. Milner, *Communicating Sequential Processes* (CSP, 1978) de T. Hoare. C'est la famille la plus expressive des modèles à états.

Caractéristiques d'une machine de Mealy

- **non hiérarchique**
- un état dénote l'état complet du système
- le système est dans **un seul état à la fois**
- une **transition est atomique** ; elle ne peut être décomposée.

Chapitre 4

Langages non-contextuels (Type 2)

4.1 Langages non-contextuels - grammaires et automates à pile

Ouvertures sur les transducteurs, les analyseurs, etc.

